Final Project Report

Teaching a Neural Network to Predict How an Absorption Chiller Behaves Over Time

An LSTM-based digital twin of a single-effect water / lithium-bromide chiller: data, model, evaluation and deployment in the browser

Submitted by: _____

Roll number: _____

Supervisor: _____

Department of Chemical Engineering
Indian Institute of Technology Jodhpur
7th Semester · November 2026

Abstract

Absorption chillers turn heat into cooling, but they respond slowly: after a change in the hot water or the weather they need half an hour or more to settle. Predicting that slow, history-dependent behaviour is a sequence-learning problem, and this project treats it as one.

We built a long short-term memory (LSTM) network that reads only the seven water-side inputs an operator can measure and predicts sixteen internal and external quantities of an 80 kW single-effect water / lithium-bromide chiller, one minute at a time, without ever being shown the true outputs. Because no measured data were available, the network was trained on 197 runs (about 810 hours of plant time) produced by a physics-based dynamic model that we calibrated against a corrected Aspen Plus flowsheet.

On 34 test runs it had never seen, an ensemble of three LSTMs reached a mean R^2 of 0.9991 and a mean error of 0.38 % of each output's range, about five times better than gradient boosting with hand-made history features and sixteen times better than a model without memory. We also checked what the network had learned: which inputs it relies on, whether its trends follow the physics, whether its error grows over a 12-hour day, and whether disagreement between the three networks warns of larger errors. Finally, the trained network was rewritten in JavaScript and now runs live in a public web dashboard with a 3D view of the plant.

Keywords: LSTM, recurrent neural network, digital twin, surrogate model, time-series prediction, ensemble, absorption chiller, transient behaviour.

Contents

Abstract 2

Contents 2

1. Introduction 3

2. The prediction problem 3

3. Building the training data 5

4. The models 7

5. Training 8

6. Results on unseen test runs 8

7. Looking inside the model 11

8. From trained model to working digital twin 13

9. Discussion 15

10. Conclusions 16

References 16

1. Introduction

A digital twin is a model of a machine that runs alongside it and answers "what will happen if…" faster than the machine itself. For an absorption chiller, the useful questions are about time: if the sun goes behind a cloud and the hot water cools by 10 K, how far will the cooling output drop, how fast, and will the salt solution come close to crystallising on the way? Steady-state process simulators such as Aspen Plus cannot answer this, because they only describe the end point. Detailed dynamic simulators can, but they are slow to set up and to run, and are rarely available in the control room.

Machine learning offers a third route. A neural network trained on enough realistic examples can learn the input-to-output behaviour of the plant, including its slow memory, and then predict hours of operation in a fraction of a second. This report is about building such a network properly: generating training data the network can trust, choosing a model that can represent memory, comparing it fairly against simpler alternatives, examining what it actually learned, and putting it to use.

The objectives were:

- to produce a large, physically checked dataset of chiller transients, since no plant measurements were available;
- to train a model that predicts the chiller's behaviour over time from the measurable inputs alone;
- to show that the model's memory matters, by comparing it with strong models that have no memory or only a fixed one;
- to test the model beyond accuracy numbers: stability, trends, uncertainty and the inputs it relies on;
- to deploy the model so that anyone can use it in a web browser.

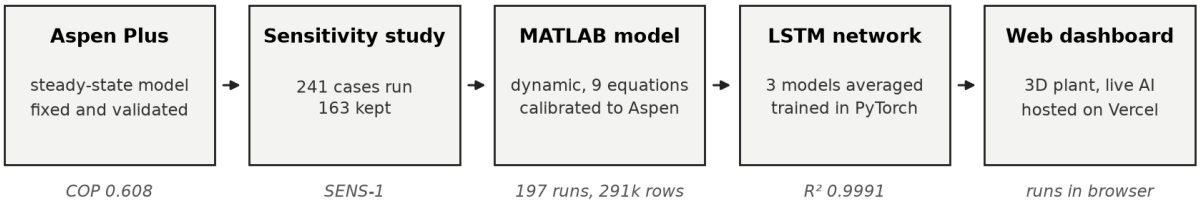


Figure 1. The project at a glance. The first three stages exist to give the network good data; the last two are the AI work that this report focuses on.

2. The prediction problem

2.1 The machine in brief

A single-effect absorption chiller uses water as the refrigerant and a concentrated lithium-bromide (LiBr) solution to absorb it. Hot water at about 105 °C boils water vapour out of the solution in the generator. The vapour condenses, drops to a deep vacuum and boils again at about 2 °C in the evaporator, cooling the building's chilled water from 12 °C to 7 °C. The solution soaks the vapour back up in the absorber, and a small pump returns it to the generator. Cooling water from a tower removes the heat from the absorber and the condenser.

For the AI, the chiller is a box with seven inputs (the temperature and flow of the hot, cooling and chilled water, and the solution pump flow) and many internal states that we want to know. Figure 2 shows the Aspen Plus flowsheet that defined the plant's design point; the details of correcting it are kept short here (Section 3.1).

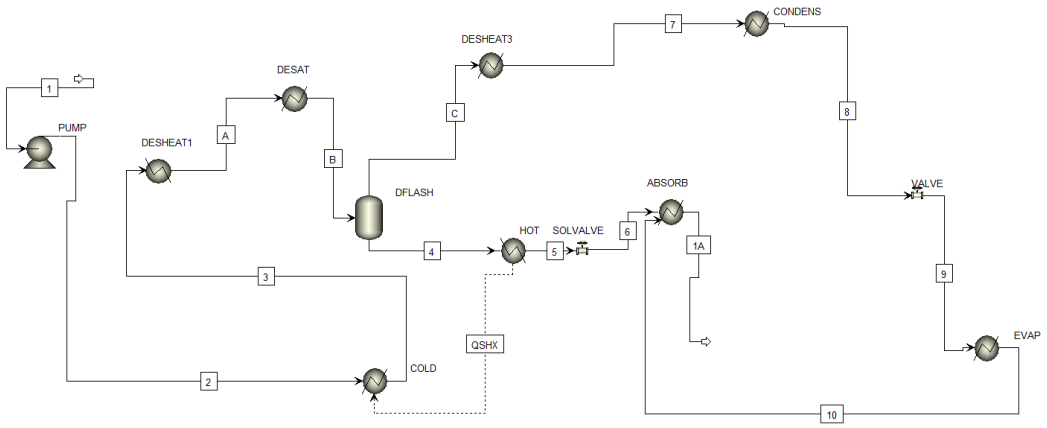


Figure 2. Aspen Plus flowsheet of the chiller (screenshot from the software).

2.2 Why it needs a model with memory

Figure 3 shows the difficulty. After the hot water is raised by 10 K, the heat input jumps at once, but the cooling output takes about 18 minutes to cover 63 % of its change and about 45 minutes to settle. The salt concentration moves just as slowly. At the moment of the step, the inputs are already at their new values while the outputs are still near the old ones, so the same inputs can correspond to very different outputs depending on what happened before. A model that sees only the present inputs cannot tell these situations apart; it needs memory of the past.

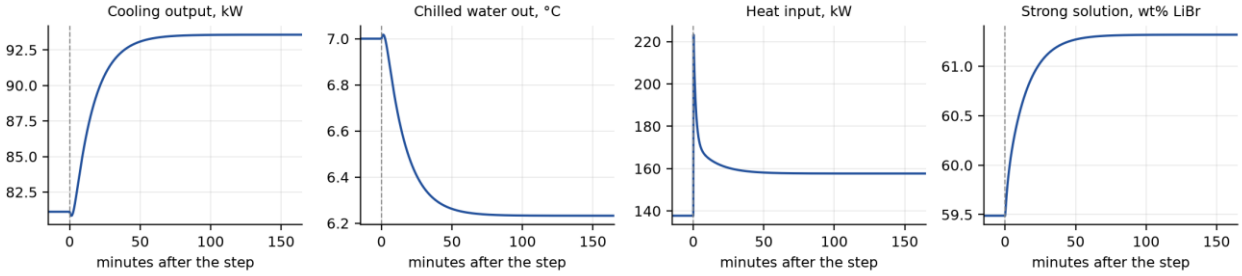


Figure 3. Response of the dynamic model to a 10 K rise in hot-water temperature at time zero. The heat input reacts at once; everything else lags by tens of minutes.

2.3 What exactly the model must do

We set the task in its hardest and most useful form, often called simulation mode. The network receives the seven inputs every minute and must output sixteen quantities (Table 1). It is never given the true value of any output, not even at the start, so it cannot correct itself from measurements; it has to keep its own internal picture of the plant for hours at a time. A model that passes this test can be used for planning and "what-if" studies, where the future outputs are by definition unknown.

Table 1. Inputs the model receives and outputs it predicts.

	Quantities
7 inputs	hot-water inlet temperature and flow; cooling-water inlet temperature and flow; chilled-water return temperature and flow; solution pump flow
16 outputs	cooling output (evaporator duty); chilled-water outlet temperature; heat input (generator duty); condenser and absorber duties; hot- and cooling-water outlet temperatures; generator, condenser, evaporator and absorber temperatures; strong and weak solution concentrations; high and low pressures; refrigerant flow
derived	COP averaged over 30 minutes, computed from the predicted duties (not learned separately)

3. Building the training data

Every machine-learning model is limited by its data. We had no measurements from a real chiller, so the data had to come from simulation, and the simulation had to be trustworthy. This took three steps.

3.1 A corrected steady-state reference

The starting Aspen Plus model gave a COP of only 0.21 and an energy balance that was off by more than 42,000 kcal/h. Recalculating every block from the stream enthalpies showed that the flash drum was silently condensing the vapour the generator had just produced, the solution loop did not close, and a preheater was cooling instead of heating. Seven specification changes, with no blocks added, closed the energy balance within 0.05 % and gave a COP of 0.608, which rises with generator temperature as it should. A sensitivity study of 241 cases (163 free of crystallisation risk) then gave a map of steady operating points to check the dynamic model against.

3.2 A dynamic "teacher" model

Following Kohlenbach and Ziegler (2008), we wrote a lumped dynamic model in MATLAB: the generator, condenser, evaporator and absorber are each one well-mixed volume with its own metal mass, temperature and liquid inventory, giving nine differential equations. We added the three external water circuits and calibrated the heat-transfer coefficients so that the model settles exactly on the corrected Aspen design point. Its steady states agree with all 163 Aspen cases (COP correlation 0.9998). This model is the "teacher": the network learns to imitate it.

3.3 Designing the scenarios

A network only learns what it is shown, so the scenarios were chosen to cover both the kind of changes a real chiller sees and the full input space. Most runs are random sequences, in which all seven inputs jump to new random values every 5 to 30 minutes. These "amplitude-modulated pseudo-random" signals are a standard tool in system identification because they excite fast and slow behaviour at the same time. Steps, ramps, start-ups and shut-downs add the clean, structured events, and five 12-hour daily profiles imitate a solar-driven day.

Table 2. Scenarios simulated with the dynamic model.

Type	Runs	Length	What changes
Random sequences	149	4 h	all seven inputs change at random every 5–30 min
Steps	19	3 h	one input jumps suddenly
Ramps	16	4 h	one input changes gradually over 10–60 min
Start-up / shut-down	8	4 h	hot water rises from 80 to 105 °C, or falls back
Daily profiles	5	12 h	solar-type day with changing weather and cooling load

Each run was checked automatically for crystallisation, freezing in the evaporator and energy conservation; two runs failed and were removed. The 197 remaining runs contain 291,437 rows at 10-second intervals, and the worst energy-balance error in any run is 0.03 %.

3.4 Splitting the data without leaking it

Neighbouring time steps are almost identical, so splitting the rows at random would let the network "see" the test data through its neighbours and would give a falsely good score. We therefore split by whole runs: 134 for training, 29 for validation (used to choose the best training epoch) and 34 for testing, which were not touched until the end. All five daily profiles were placed in the test set on purpose, so that the final test includes a kind of operation the network never trained on.

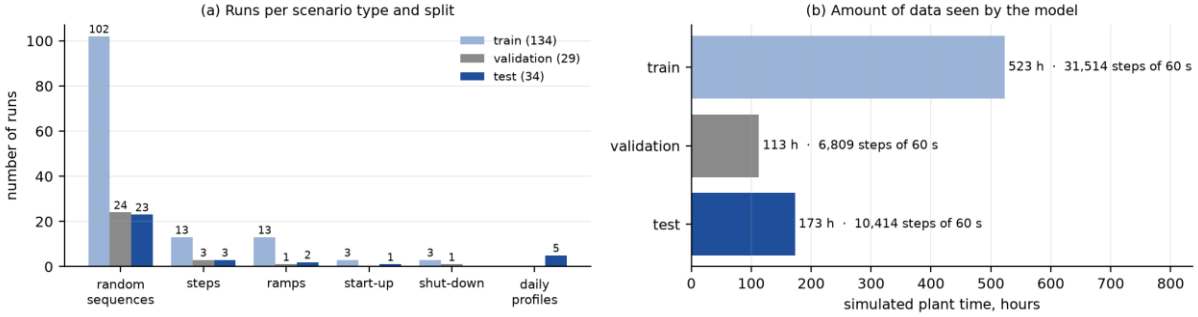


Figure 4. How the 197 runs were divided. All daily profiles are held out for testing.

Figure 5 compares the distribution of each input in the training and test sets. The test inputs lie inside the training range everywhere, so the test measures interpolation within known conditions, which is what the model will be used for, and not extrapolation.

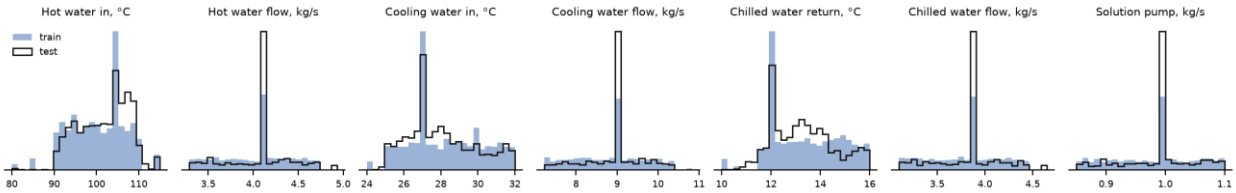


Figure 5. Distribution of the seven inputs in the training runs (filled) and the test runs (outline).

3.5 Preparing the sequences

- **Time step.** The plant's time constants are 18–47 minutes, so we kept one row per minute (60 s). This keeps all the dynamics while making the sequences six times shorter, which made training on a laptop CPU practical.
- **Scaling.** Each input and output was converted to a z-score using the mean and standard deviation of the training runs only, so no information from the test set leaks into the model.
- **Warm-up.** Every run starts from a steady state. A recurrent network, however, starts with an empty memory. We therefore feed 60 minutes of the first input row before each run and throw away the predictions from that hour, which lets the network's memory settle to the starting steady state (Section 7.4).
- **Batching.** Runs have different lengths (3 to 12 hours). They are padded to the same length, and a mask makes sure the padded steps do not count in the loss.

4. The models

To show what memory is worth, we trained three kinds of model on the same data and judged them on the same test runs.

4.1 Baseline 1: gradient boosting without memory

Gradient-boosted decision trees are among the strongest general-purpose models for tabular data. We trained one regressor per output (scikit-learn's HistGradientBoostingRegressor, 600 trees, learning rate 0.08, 48 leaves) on the current seven inputs only. This model can learn the steady-state map of the chiller very well, but it has no idea of time.

4.2 Baseline 2: gradient boosting with input history

The usual way to give a static model some memory is to engineer features from the past. For every input we added its values 1, 2, 5, 10, 20 and 40 minutes ago and four first-order filtered versions with time constants of 5, 15, 30 and 60 minutes, which mimic how a tank of liquid smooths a change. This gives 77 features per time step. It is a strong, physics-inspired baseline, close to the classical NARX and Hammerstein models used in process control.

4.3 The LSTM network

A long short-term memory network (Hochreiter and Schmidhuber, 1997) is a recurrent neural network: it processes the sequence one step at a time and carries a memory vector from each step to the next. Its key feature is a set of gates that decide, at every step, how much of the old memory to keep, how much new information to write and how much to show at the output. For an input x_t and the previous hidden state h_{t-1}, the cell computes

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \qquad i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \qquad o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_g \cdot [h_{t-1}, x_t] + b_g) \qquad h_t = o_t \odot \tanh(c_t)$$

where σ is the logistic function and $\odot$ is element-wise multiplication. The cell state c_t is the long-term memory: when the forget gate f_t stays close to one, information survives for many steps, which is exactly what a chiller with a 45-minute settling time requires. In our setting the memory ends up playing the role of the stored heat and salt in the vessels, even though the network was never told about them.

Our network (Figure 6) has two stacked LSTM layers of 128 cells, followed by a small dense head (128 units, tanh, then 16 outputs). It has about 220,000 trainable parameters. We trained three copies that differ only in their random starting weights (seeds 0, 1 and 2) and average their predictions. Such an ensemble usually reduces error a little and, more usefully, the disagreement between its members gives an estimate of uncertainty (Section 7.3).

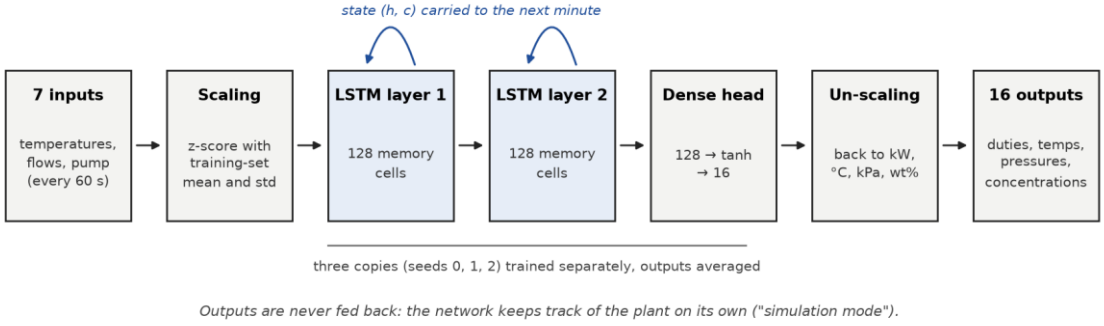


Figure 6. The LSTM digital twin. The curved arrows are the memory passed from one minute to the next.

We first tried a GRU, a lighter relative of the LSTM. On our CPU it took 14 to 76 seconds per epoch against 1.4 to 2.4 seconds for the LSTM in PyTorch, so we chose the LSTM to be able to train longer and repeat the training with three seeds.

5. Training

Each network sees whole runs from beginning to end and is trained by back-propagation through time, so the error at hour three can adjust how the network stored information at minute five. Table 3 lists the settings.

Table 3. Training settings of each LSTM.

Setting	Value
Loss	mean squared error on the scaled outputs, padded steps masked out
Optimiser	AdamW, learning rate 3×10^{-3}, weight decay 1×10^{-5}
Learning-rate schedule	cosine decay to 1/50 of the start value over 600 epochs
Gradient clipping	total norm limited to 1.0 (keeps recurrent training stable)
Batch	32 complete runs
Model selection	validation loss checked every 5 epochs; the best checkpoint is kept
Hardware and time	laptop CPU, 8 threads; about 30–47 min per network, 142 min including the baselines

Figure 7 shows the learning curves. The loss falls by about two orders of magnitude, quickly at first and then slowly as the learning rate decays. Training and validation losses stay close throughout, so the networks are not over-fitting; the best validation losses of the three seeds (0.00095, 0.00092 and 0.00094) are nearly the same, which shows the result does not depend on a lucky initialisation. The short bumps in the early epochs are typical of recurrent networks at a high learning rate and disappear as the rate decays.

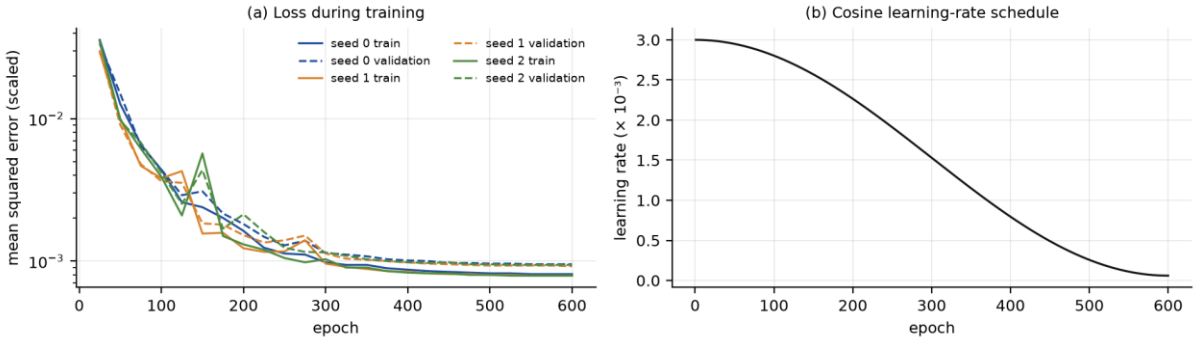


Figure 7. Training (solid) and validation (dashed) loss for the three seeds, and the learning-rate schedule.

6. Results on unseen test runs

6.1 Comparison of the models

All numbers in this section are for the 34 test runs (173 hours of plant time) that were kept aside during training. We report the coefficient of determination R^2 and the root-mean-square error as a percentage of each output's range (NRMSE), averaged over the seventeen outputs.

Table 4. Accuracy of the four models on the 34 test runs.

Model	Mean R^2	Worst R^2	Mean NRMSE
Gradient boosting, current inputs only	0.745	−0.663	6.09 %
Gradient boosting with input history	0.978	0.931	2.07 %
LSTM, single network	0.9991	0.9953	0.39 %
LSTM, ensemble of three (final model)	**0.9991**	**0.9954**	**0.38 %**

The model without memory is clearly not enough. It does reasonably on outlet water temperatures, which follow the inputs closely, but fails badly on the 30-minute COP (R^2 below zero, worse than predicting the average) and on the absorber duty and concentrations, which are dominated by stored heat and salt. Adding input history recovers most of the behaviour, but the LSTM is still five times more accurate on average and better on every single output (Figure 8).

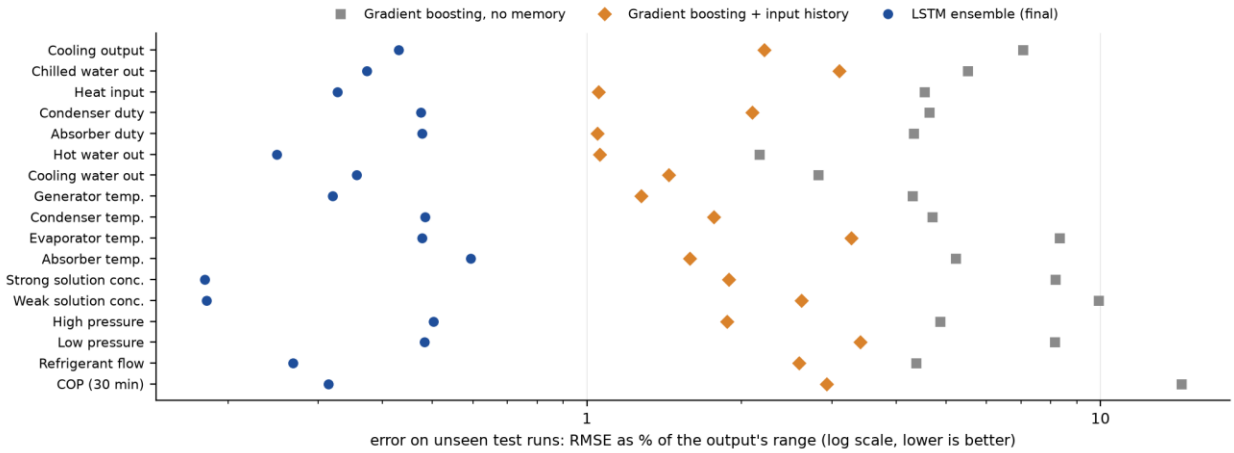


Figure 8. Error of each model for every output on the unseen test runs (log scale).

Figure 9 shows why. After a single step (top row), the history-based model gets the shape roughly right but settles at the wrong level, because a fixed set of past values cannot represent a response whose speed depends on the state of the plant. The LSTM's learned memory has no such limit and follows the truth almost exactly, both after the step and through a full day.

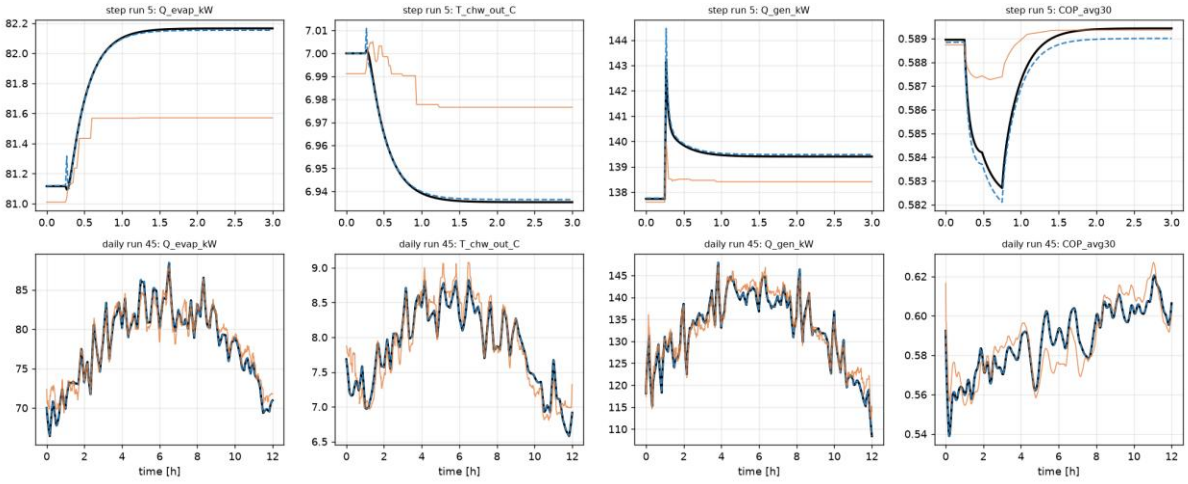


Figure 9. Two unseen test runs: a single step (top) and a full day (bottom). Black: dynamic model; blue dashes: LSTM ensemble; orange: gradient boosting with input history.

6.2 Accuracy of the final model, output by output

Table 5. Accuracy of the LSTM ensemble for each output on the test runs.

Output	Unit	R^2	RMSE	NRMSE %
Cooling output	kW	0.9990	0.31	0.43
Chilled water out	°C	0.9996	0.024	0.37
Heat input	kW	0.9984	0.77	0.33
Condenser duty	kW	0.9991	0.35	0.47
Absorber duty	kW	0.9954	1.01	0.48
Hot water out	°C	0.9998	0.077	0.25
Cooling water out	°C	0.9997	0.030	0.36
Generator temperature	°C	0.9996	0.087	0.32
Condenser temperature	°C	0.9993	0.063	0.48
Evaporator temperature	°C	0.9993	0.039	0.48
Absorber temperature	°C	0.9991	0.049	0.59
Strong solution	wt%	0.9998	0.012	0.18
Weak solution	wt%	0.9999	0.010	0.18
High pressure	kPa	0.9992	0.033	0.50
Low pressure	kPa	0.9993	0.0023	0.48
Refrigerant flow	kg/s	0.9997	0.00008	0.27
COP over 30 min (derived)	–	0.9992	0.0014	0.31

In practical terms, the typical error is 0.3 kW on a cooling output of about 80 kW, 0.02 °C on the chilled-water temperature and 0.01 percentage points on the salt concentration, well below what plant instruments resolve. The weakest output is the absorber duty. It has sharp spikes for a minute or two after the hot water drops (Figure 15a), and getting the exact height of a one-minute spike is hard at a 60-second step. The parity plots in Figure 10 show no systematic bias: the points lie on the diagonal across the whole range.

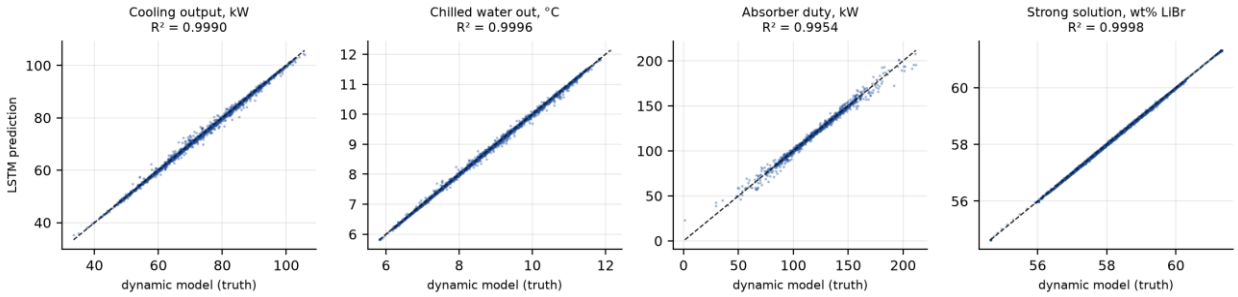


Figure 10. Predicted against true values for every minute of every test run.

6.3 A full day the model had never seen

The daily profiles are the closest thing to real operation in the dataset, and none of them was used in training. Figure 11 shows one: the inputs drift through a 12-hour day and the LSTM follows the cooling output, chilled-water temperature and heat input so closely that the two lines are hard to separate. Over the five days, the mean R^2 is 0.99993, the cooling-output error is 0.04 kW and the chilled-water error is 0.004 °C.

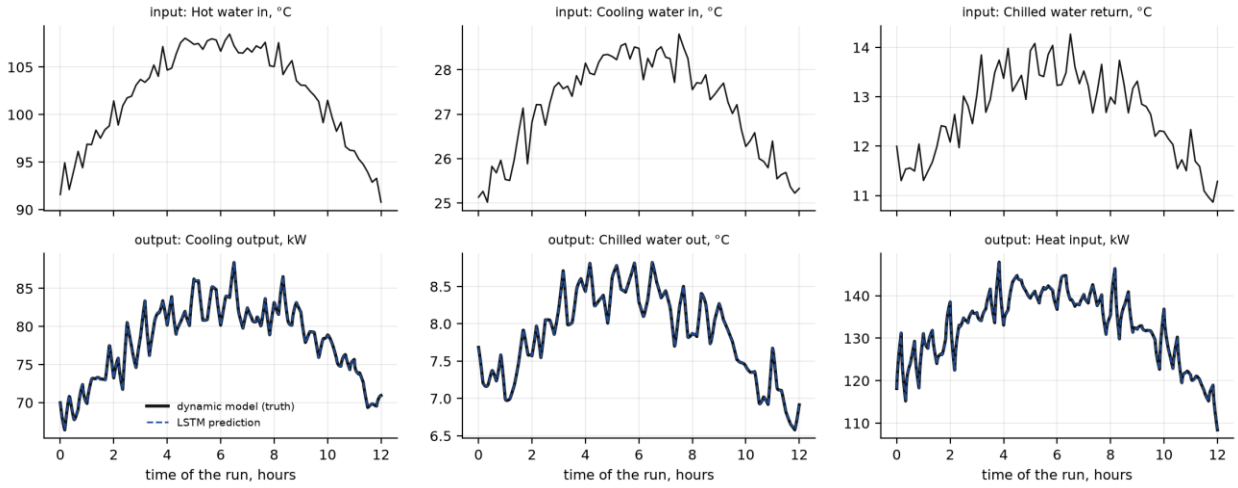


Figure 11. Test run 45, a 12-hour daily profile. Top: three of the seven inputs. Bottom: truth and prediction.

6.4 Where the errors come from

Breaking the error down by kind of run (Figure 12a) shows that the random sequences are the hardest, with a mean error of 0.52 % of range, while steps, ramps and daily profiles are all at 0.15 % or below. This makes sense: the random runs change all seven inputs at once, every few minutes, and are harsher than anything a real chiller sees.

In simulation mode, small errors could in principle pile up over time. Figure 12b checks this on the five 12-hour test days: the median and 90th-percentile errors of the cooling output stay flat or fall slightly over the day. The model does not drift, because its memory is anchored to the inputs rather than to its own past outputs.

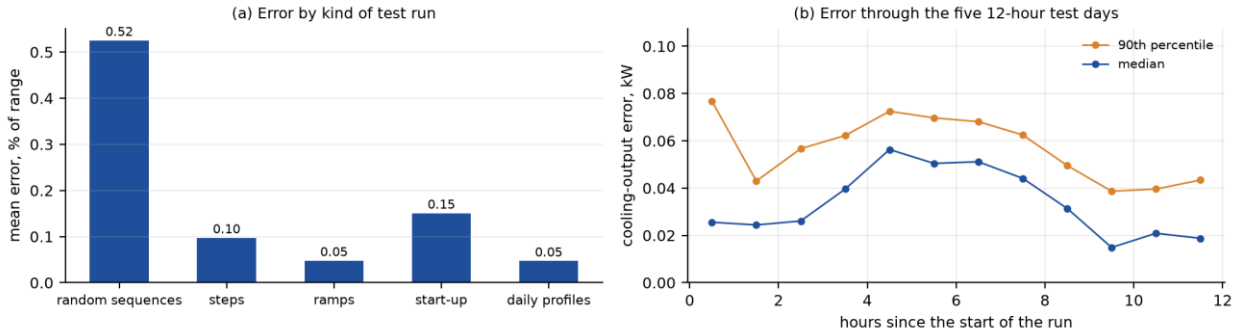


Figure 12. Error by kind of test run (a) and cooling-output error through the five unseen daily profiles (b).

7. Looking inside the model

A high R^2 says the model fits the test data; it does not say the model is sensible. We ran four further checks to see whether the network has learned behaviour that an engineer would recognise.

7.1 Which inputs matter

To measure how much the network relies on each input, we froze one input at its starting value for the whole of every test run, ran the ensemble again and recorded how much the mean error grew. Figure 13 shows the result. The hot-water temperature matters most by far (error up by 19 percentage points), followed by the cooling-water temperature (10) and the chilled-water return temperature (7). The four flows matter about ten times less.

This is the same ranking a chiller engineer would give: the three temperatures set the three pressure levels and hence the whole cycle, while the flows only change heat-transfer rates within each exchanger. The network has worked this out from data alone.

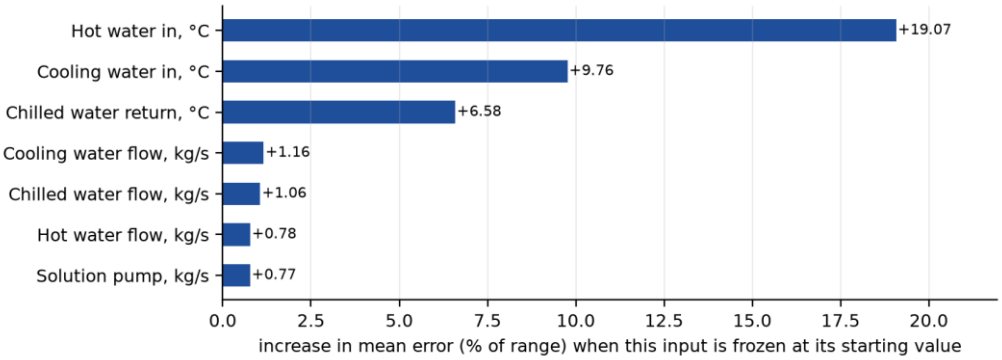


Figure 13. Increase in mean test error when each input is held constant instead of following the run.

7.2 Do the learned trends follow the physics?

We then used the network as a virtual test rig. Starting from the design point, we changed one temperature at a time across its training range, let the network settle, and read off the cooling output and the COP (Figure 14).

- Hotter driving water gives more cooling (48 to 94 kW from 80 to 115 °C) and a higher COP, with the COP gain flattening at high temperature, as it does in real single-effect machines.
- Warmer cooling water gives less cooling (91 to 65 kW from 24 to 32 °C) and a lower COP, because it raises both the condenser pressure and the absorber temperature.
- A warmer chilled-water return gives more cooling and a higher COP, because the evaporator can work at a higher pressure.

All three trends are smooth and have the right sign, and the design point predicts 81.1 kW, matching the dynamic model's 81 kW. None of these curves was shown to the network directly; they emerge from what it learned about transients.

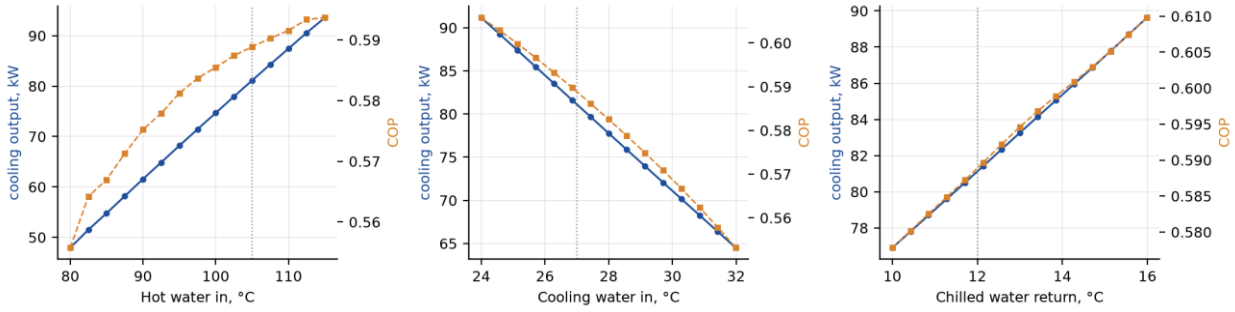


Figure 14. Steady-state cooling output (blue) and COP (orange) predicted by the network as one temperature is varied. Dotted lines mark the design point.

7.3 Can the model tell when it is unsure?

The three networks in the ensemble agree closely almost all the time, so in Figure 15a the band between them is barely wider than the line. They do not agree equally everywhere, though. We sorted every test minute by the spread between the three networks and compared it with the actual error of the ensemble (Figure 15b). The tenth of minutes with the largest spread has about 20 times the error of the tenth with the smallest spread, and the correlation between spread and error is 0.66.

This gives the dashboard a simple, honest warning light: when the three networks disagree, the prediction should be trusted less. Averaging also helps accuracy slightly, from 0.395 % for a single network to 0.386 % for the ensemble on the sixteen predicted outputs.

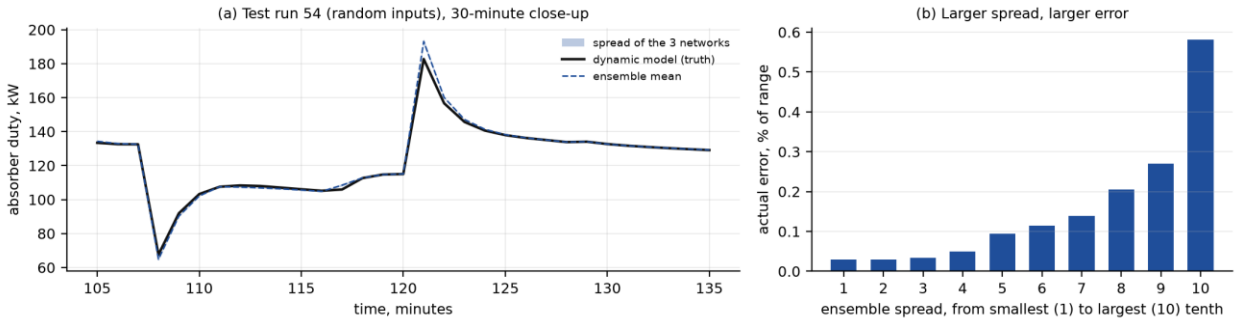


Figure 15. Ensemble behaviour. (a) A 30-minute close-up of an absorber-duty spike in a random test run. (b) Mean error for each tenth of the test minutes, sorted by how much the three networks disagree.

7.4 Warm-up and long-run stability

A recurrent model used as a digital twin must start cleanly and must not wander off when nothing is happening. Figure 16a starts the ensemble from an empty memory at the design inputs: the prediction overshoots briefly and is within 0.02 kW of its final value after about 40 minutes. The 60-minute warm-up described in Section 3.5 covers this with margin. Figure 16b then keeps the inputs constant for 50 hours, four times longer than any training run; the cooling output changes by only 0.003 kW. The network has learned a stable steady state rather than a pattern that only holds for the length of the training runs.

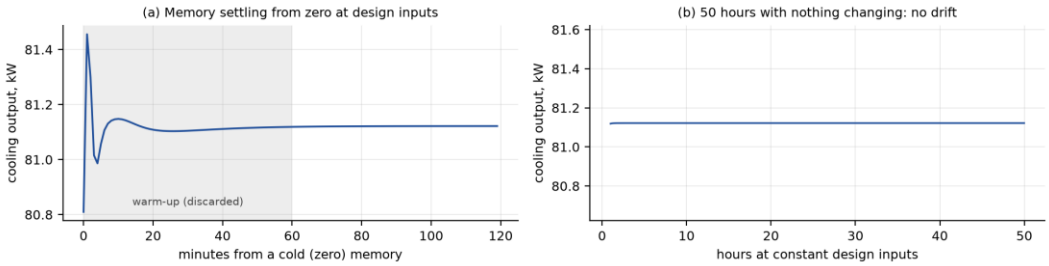


Figure 16. Start-up of the network's memory (a) and a 50-hour run at constant design inputs (b).

8. From trained model to working digital twin

8.1 Running the network in a web browser

A model that only runs inside a Python notebook is of little use to an operator. We therefore exported the weights of the three networks and the scaling constants to a portable file and wrote the LSTM forward pass again in plain JavaScript (about 100 lines, no libraries). The weights travel inside the web page as base64-encoded 32-bit floats, so the page works without any server. Every minute of plant time, the page scales the seven inputs, runs both LSTM layers and the dense head for each of the three networks, and returns the mean and the minimum and maximum of the members.

We verified the JavaScript engine against PyTorch on a full 12-hour test run (721 steps): the largest difference over all sixteen outputs was 0.0014 kW, which is ordinary 32-bit rounding. One minute of plant time takes about 1.5 ms in the browser for the whole ensemble, so a 12-hour day can be simulated in about a second.

8.2 The dashboard

The dashboard is built like the control screen of a process plant. A 3D model of the chiller, laid out like the Aspen flowsheet, shows the flow in every pipe and the live temperature, pressure and flow of every stream, all coming from the network. The user can pick a situation (a sunny day, a fall in hot-water temperature, a hot afternoon at the cooling tower, a start-up) or move the seven inputs directly and watch the plant respond. Warnings appear when the predicted salt concentration nears crystallisation or the evaporator nears freezing.

A separate validation page replays five of the test runs with the network running live next to the dynamic model's results, so anyone can check the accuracy claims in this report for themselves. Streams that the network does not predict directly, such as the solution leaving the heat exchanger, are derived from the predicted values with simple, documented relations and are marked as approximate.

The dashboard is public at libr-chiller-twin.vercel.app. The full project, including data, training code, trained models, figures and documentation, is on GitHub at github.com/darshanprajapati07/Absorption_chiller, and every figure in this report can be regenerated from the scripts in its Report folder.

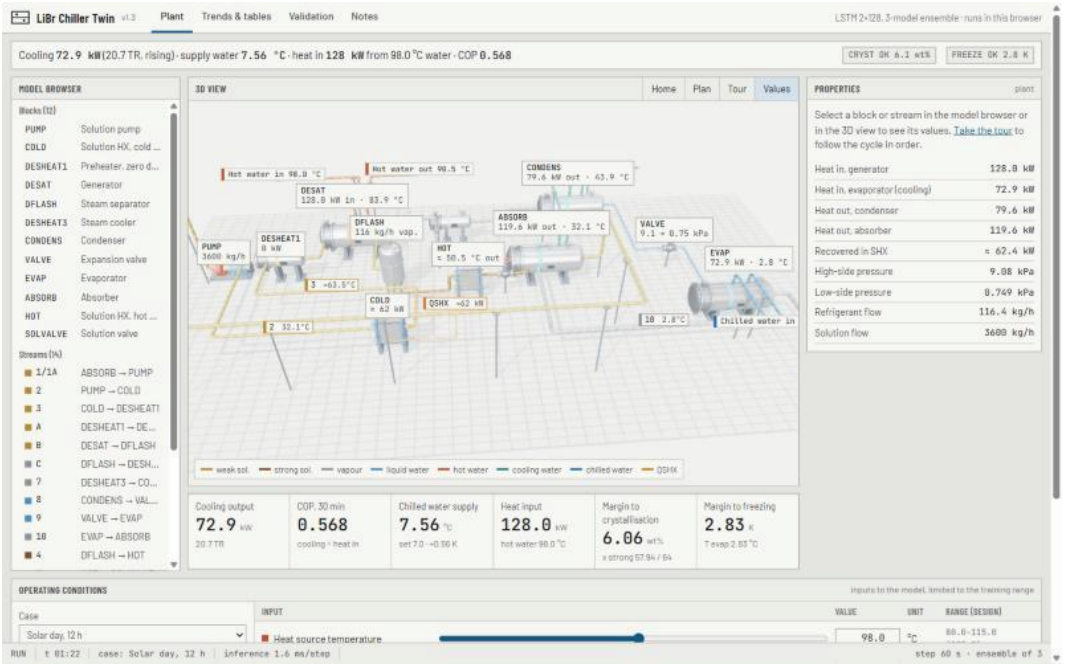


Figure 17. Plant screen of the dashboard: model browser, 3D view, properties and live readings predicted by the LSTM.



Figure 18. The generator selected in the 3D view, with the live readings below.

9. Discussion

9.1 What made the difference

- **Memory in the model, not in the features.** The history-based baseline had 77 well-chosen features and still lost to the LSTM on every output. Hand-made lags fix the time scales in advance; the LSTM learns them, and learns that they change with the state of the plant.
- **Data designed for learning.** Random multi-input sequences taught the network how the inputs interact, while steps and daily profiles tested it on realistic events. Splitting by whole runs kept the test honest.
- **Simulation mode from the start.** Training the network to run without seeing any outputs made it harder to train, but it is the reason the model can run for 50 hours without drifting and can be used for planning.
- **Checking more than accuracy.** The input ranking, the steady-state trends and the stability test give confidence that the network has learned the chiller and not a quirk of the dataset.

9.2 Limitations

The most important limitation is that the network has learned from a simulator, not from a real machine. It is as good as the dynamic model, and that model's metal masses and liquid hold-ups were estimated by scaling a published 10 kW unit; they control how fast the chiller responds. The near-perfect scores in this report therefore measure how well the network imitates its teacher, not how well it would predict a real chiller.

The model is also only reliable inside the range of conditions used in training, and runs that crystallised or froze were removed, so it can warn when the plant approaches those limits but cannot describe what happens beyond them. Finally, the 60-second step smooths very short events, which is why the absorber-duty spikes are the hardest output to match.

9.3 Next steps

- **Transfer learning to a real chiller.** Pre-train on simulation as done here, then fine-tune on a few days of plant measurements. This usually needs far less real data than training from scratch.
- **Physics-informed training.** Add the energy and salt balances to the loss so that predictions conserve energy exactly, which would also help outside the training range.
- **State estimation.** Feed occasional measurements back into the network's memory to correct drift on a real plant.
- **Model predictive control.** Because one prediction step takes milliseconds, the network is fast enough to sit inside an optimiser that adjusts the hot-water flow to hold 7 °C chilled water while staying clear of crystallisation.

10. Conclusions

- A two-layer LSTM ensemble, given only the seven water-side inputs, predicts sixteen quantities of an absorption chiller minute by minute with a mean R^2 of 0.9991 on 34 unseen test runs, and 0.9999 on five unseen 12-hour days.
- Memory is essential: a strong model without memory reached only R^2 0.745, and one with engineered history features 0.978, with five times the LSTM's error.
- The network's behaviour is physically sensible: it relies most on the three water temperatures, reproduces the correct steady-state trends, does not drift over 50 hours, and the disagreement within the ensemble flags its less reliable predictions.
- Reliable training data could be produced without plant measurements by correcting the Aspen model and calibrating a dynamic model to it.
- The trained network runs in an ordinary web browser in about 1.5 ms per minute of plant time, making an interactive digital twin available to anyone with a link.

References

[1] Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

[2] Goodfellow, I., Bengio, Y. and Courville, A. (2016). Deep Learning, chapter 10: Sequence modeling: recurrent and recursive nets. MIT Press.

[3] Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization (AdamW). International Conference on Learning Representations.

[4] Lakshminarayanan, B., Pritzel, A. and Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. Advances in Neural Information Processing Systems 30.

[5] Ke, G. et al. (2017). LightGBM: a highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems 30. (Basis of scikit-learn's histogram gradient boosting.)

[6] Ljung, L. (1999). System Identification: Theory for the User, 2nd ed. Prentice Hall.

[7] Kohlenbach, P. and Ziegler, F. (2008). A dynamic simulation model for transient absorption chiller performance. Part I: The model. International Journal of Refrigeration, 31, 217–225.

[8] Herold, K.E., Radermacher, R. and Klein, S.A. (2016). Absorption Chillers and Heat Pumps, 2nd ed. CRC Press.

[9] Paszke, A. et al. (2019). PyTorch: an imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems 32.

[10] AspenTech. Aspen Plus user documentation: ELECNRTL property method.